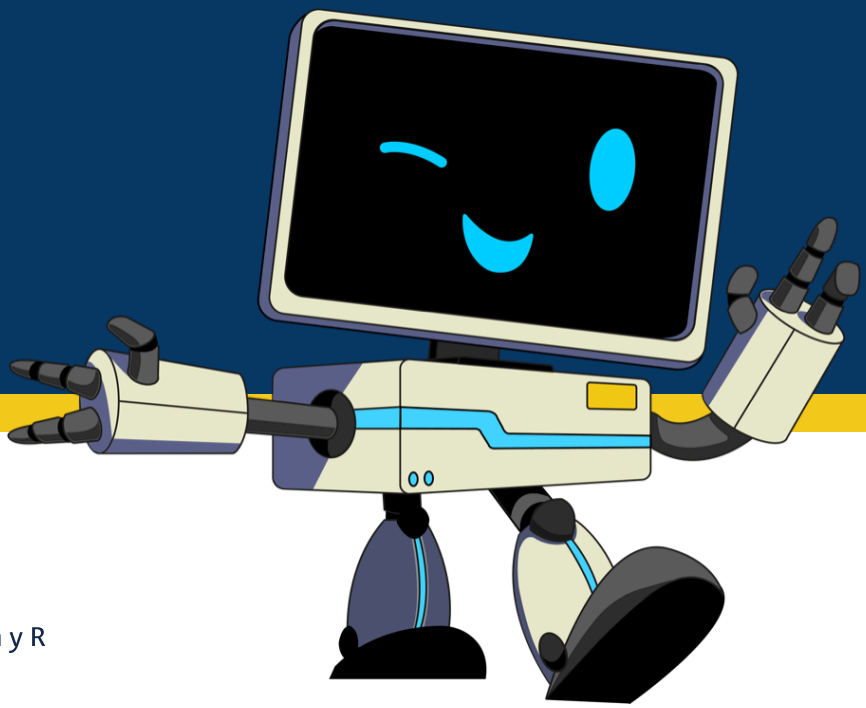
GUÍA DE REPASO · SESIÓN 1

¿Qué es programar? Primeros pasos con Python

Definiciones, ejemplos y ejercicios para repasar lo que vimos en clase.



CURSO LIBRE

Introducción a la Programación en Python y R

SEMESTRE

2026-2

ESTUDIANTE AUXILIAR

Yony Sebastián Chaparro Mesa

MATERIAL DEL CURSO

Moodle FCE

UIFCE
UNIDAD DE INFORMÁTICA

Unidad de Informática · Facultad de Ciencias Económicas
Universidad Nacional de Colombia, Sede Bogotá
uniic_bog@unal.edu.co

Cómo usar esta guía

Esta guía resume la primera sesión del curso. Cada tema tiene **definiciones** (recuadros azules), **ejemplos de código** con su salida y **advertencias** sobre los errores más comunes (recuadros amarillos).

La mejor forma de repasar es con Google Colab abierto: **escribe tú mismo cada ejemplo**, ejecútalo y compara tu salida con la de la guía. Leer código ayuda, pero escribirlo es lo que hace que se quede.

TEMA	QUÉ DEBERÍAS PODER HACER AL TERMINAR
1. ¿Qué es programar?	Explicar qué es un programa, un algoritmo y un lenguaje de programación.
2. Google Colab	Abrir un notebook, escribir código y ejecutar celdas.
3. `print()`	Mostrar texto y resultados en pantalla.
4. Variables	Guardar datos con un nombre y usarlos después.
5. Tipos de datos	Reconocer `int`, `float`, `str` y `bool`, y convertir entre ellos.
6. Operadores	Hacer cálculos con los operadores aritméticos.
7. Errores	Leer un mensaje de error e identificar dónde está el problema.
8. `input()`	Pedir datos al usuario.

Al final encontrarás un **resumen**, un **glosario**, **ejercicios** para practicar y sus **soluciones**.

1 ¿Qué es programar?

Aunque no lo notemos, damos instrucciones a máquinas todos los días: programamos una alarma, escribimos una fórmula en Excel o creamos un filtro para el correo. Programar es hacer eso mismo, pero de forma más explícita y completa.

> **DEFINICIÓN**
> **Programar** es escribir instrucciones precisas que un computador puede ejecutar para resolver un problema o hacer una tarea.

> **DEFINICIÓN**
> Un **programa** es el resultado: una lista de instrucciones que el computador sigue **en orden, de arriba hacia abajo**.

El computador es literal

El computador hace exactamente lo que le dices, no lo que quisiste decir. No supone, no adivina y no "entiende el contexto". Si una instrucción está mal escrita o falta un paso, no lo completa por su cuenta: se detiene o hace algo distinto a lo que esperabas. Piensa en una receta escrita para alguien que nunca ha cocinado y que no improvisa.

Algoritmo: la idea antes del código

> **DEFINICIÓN**
> Un **algoritmo** es una secuencia **finita y ordenada** de pasos para resolver un problema.

Un algoritmo no depende de ningún lenguaje: se puede escribir en español. Por ejemplo, para calcular el promedio de tus notas:

1. Reunir todas las notas.
2. Sumarlas.
3. Contar cuántas notas hay.
4. Dividir la suma entre la cantidad.
5. Mostrar el resultado.

Cuando ese algoritmo se escribe en un lenguaje de programación, se convierte en un programa. El camino completo es: **problema → algoritmo → código → ejecución → resultado**. El mismo algoritmo se puede escribir en Python o en R (todavía no tienes que entender cada símbolo):

Python

```python
notas = [3.0, 4.0, 3.5]
promedio = sum(notas) / len(notas)
print(promedio)   # 3.5
```

R

```r
notas <- c(3.0, 4.0, 3.5)
promedio <- mean(notas)
print(promedio)   # 3.5
```

Lenguajes de programación

> **DEFINICIÓN**
> Un **lenguaje de programación** es un idioma intermedio entre las personas y la máquina. Tiene un vocabulario (palabras reservadas, como `print` o `if`) y una gramática estricta llamada **sintaxis**.

NIVEL	EJEMPLO	QUIÉN LO ENTIENDE MEJOR
Lenguaje humano	“Calcula el promedio de mis notas”	Las personas
Alto nivel (Python, R)	`mean(notas)`	Las personas, y se traduce para la máquina
Bajo nivel (ensamblador)	`MOV EAX, [EBX]`	Más cerca de la máquina
Código máquina	`10110000 01100001`	El procesador

Python y R son lenguajes de **alto nivel**: se parecen más al idioma humano y nos dejan concentrarnos en el problema, no en los detalles del hardware.

Compilado o interpretado

> **COMPILADO**
> Traduce **todo el programa** antes de ejecutarlo, como traducir un libro completo antes de publicarlo. Ejemplos: C, Java.

> **INTERPRETADO**
> Lee y ejecuta **línea por línea**, como un intérprete en una conferencia. Ejemplos: Python, R.

Como Python es interpretado, escribes una línea y ves el resultado al instante. Por eso es ideal para aprender y para explorar datos.

¿Por qué Python y R?

- **Python** es de propósito general (datos, web, automatización) y fue diseñado para ser legible. Lo usamos en las sesiones 1 a 4, en Google Colab.
- **R** nació para la estadística y es muy fuerte en análisis y gráficos. Lo usamos en las sesiones 5 a 7, en Posit Cloud.
- Ambos son **gratuitos**, tienen comunidades enormes y son muy usados en análisis de datos.

2 Google Colab

Google Colab es un servicio gratuito de Google para escribir y ejecutar Python desde el navegador. No hay que instalar nada: solo necesitas una cuenta de Google.

> **DEFINICIÓN**
> Un **notebook** (cuaderno, archivo `.ipynb`) es un documento formado por **celdas**. Hay dos tipos: **celdas de código**, que se ejecutan, y **celdas de texto**, para títulos, notas y explicaciones.

Pasos básicos

1. Abre el enlace del notebook con tu cuenta de Google.

2. Ve a **Archivo → Guardar una copia en Drive**. Así trabajas sobre tu propia copia y tus cambios quedan guardados.
3. Escribe en una celda de código.
4. Ejecútala con **Shift + Enter** (o con el botón ▶ a la izquierda de la celda).
5. La **salida** aparece justo debajo de la celda.

Para agregar celdas usa los botones **+ Código** y **+ Texto** de la parte superior.

> **OJO**
> Colab recuerda lo que **ejecutaste**, no lo que está escrito en pantalla. Si cambias una celda y no la vuelves a ejecutar, el cambio no cuenta. Si algo se comporta raro, usa **Entorno de ejecución → Ejecutar todas** para correr el notebook de arriba hacia abajo.

3 Mostrar resultados con `print()`

> **DEFINICIÓN**
> Una **función** es una instrucción con nombre que hace una tarea. Se usa escribiendo su nombre seguido de paréntesis. `print()` es una función que **muestra en pantalla** lo que va entre sus paréntesis.

```
print("Hola, mundo")
print("Me llamo Ana")
print(2 + 3)
```

> **SALIDA**
> ```
> Hola, mundo
> Me llamo Ana
> 5
> ```

- El **texto** va entre comillas (dobles `"…"` o sencillas `'…'`).
- Los **números** van sin comillas: Python los calcula antes de mostrarlos.
- Puedes mostrar varios valores separándolos con **comas**; Python pone un espacio entre ellos.

```
print("Ana", "tiene", 20, "años")   # los valores se separan con comas
```

> **SALIDA**
> ```
> Ana tiene 20 años
> ```

Comentarios

Todo lo que va después de `#` es un **comentario**: Python lo ignora. Sirve para dejar notas que explican el código, para ti o para otras personas.

4 Variables

> **DEFINICIÓN**
> Una **variable** es como una **caja con etiqueta**: guarda un dato y lo puedes usar después por su nombre.

```
nombre = "Ana"
edad = 20
estatura = 1.65
print(nombre, "tiene", edad, "años")
```

> **SALIDA**
> ```
> Ana tiene 20 años
> ```

El signo = guarda, no compara

La instrucción `edad = 20` se lee **"guarda 20 en edad"**. No es la igualdad de las matemáticas. Python primero calcula lo que está a la derecha del `=` y luego lo guarda en la variable de la izquierda. Por eso esto es válido:

```
edad = 20
edad = edad + 1     # calcula 20 + 1 y guarda 21 en edad
print(edad)
```

> **SALIDA**
> ```
> 21
> ```

Una variable guarda **un solo valor a la vez**: al asignarle uno nuevo, el anterior se pierde.

Cómo nombrar una variable

NOMBRE	¿SIRVE?	POR QUÉ
`edad`	**Sí**	Corto y dice qué guarda.
`nota_final`	**Sí**	Usa guion bajo para separar palabras.
`edad2`	**Sí**	Puede tener números, pero no al inicio.
`2edad`	**No**	Empieza con número.
`nota final`	**No**	Tiene un espacio.
`nota-final`	**No**	Python lo lee como una resta: `nota` menos `final`.

- Usa nombres que digan **qué guardan**: `precio` es mejor que `x`.
- Python distingue mayúsculas de minúsculas: `Edad` y `edad` son **variables distintas**.
- Evita tildes y la ñ en los nombres de variables (en el texto entre comillas sí puedes usarlas).

5 Tipos de datos

Cada dato tiene un **tipo**, y el tipo define qué se puede hacer con él. Por ejemplo, los números se pueden dividir, pero un texto no.

TIPO	QUÉ GUARDA	EJEMPLO	PARA RECORDAR
`str`	Texto	`"Ana"`	Siempre entre comillas.
`int`	Número entero	`20`	Sin decimales.
`float`	Número decimal	`1.65`	Con **punto**, no con coma.
`bool`	Verdadero o falso	`True` , `False`	Con mayúscula inicial.

Averiguar el tipo con `type()`

```
print(type(20))
print(type(1.65))
print(type("Ana"))
print(type("20"))     # ¡entre comillas es texto!
print(type(True))
```

SALIDA

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'str'>
<class 'bool'>
```

OJO

`20` y `"20"` no son lo mismo: el primero es un número y el segundo es texto que se ve como un número. Las comillas cambian el tipo.

Convertir entre tipos

Las funciones `int()`, `float()` y `str()` convierten un dato a otro tipo:

```
print(int("20") + 1)          # texto a entero
print(float("3.5"))           # texto a decimal
print(str(20) + " años")       # entero a texto
print(int(3.9))               # int() corta los decimales, no redondea
```

SALIDA

```
21
3.5
20 años
3
```

6 Operadores aritméticos

DEFINICIÓN

Un **operador** es un símbolo que hace una operación con uno o más valores. Una **expresión** es una combinación de valores, variables y operadores que Python calcula para obtener un resultado, como `precio * cantidad`.

SÍMBOLO	OPERACIÓN	EJEMPLO	RESULTADO
`+`	Suma	`7 + 2`	`9`
`-`	Resta	`7 - 2`	`5`
`*`	Multiplicación	`7 * 2`	`14`
`/`	División	`7 / 2`	`3.5`
`//`	División entera (sin decimales)	`7 // 2`	`3`
`%`	Residuo de la división	`7 % 2`	`1`
`**`	Potencia	`7 ** 2`	`49`

- `/` siempre da un `float`, aunque la división sea exacta: `10 / 2` da `5.0`.
- `%` sirve, por ejemplo, para saber si un número es par: si `n % 2` da `0`, `n` es par.
- Para potencias se usa `**`, no `^`.

Orden de las operaciones

Python sigue el mismo orden que las matemáticas: primero potencias, luego multiplicaciones y divisiones, y al final sumas y restas. Los paréntesis cambian el orden.

```
print(2 + 3 * 4)       # primero 3 * 4
print((2 + 3) * 4)     # primero el paréntesis
```

SALIDA

```
14
20
```

Ejemplo: el total de una compra

```
precio = 12000
cantidad = 3
total = precio * cantidad
print("Total a pagar:", total)
print("Total con IVA:", total * 1.19)    # IVA del 19 %
```

SALIDA

```
Total a pagar: 36000
Total con IVA: 42840.0
```

Si cambias `precio` y vuelves a ejecutar la celda, el total se actualiza solo. Esa es una de las ventajas de usar variables.

Operadores con texto

Con texto, `+` **une** (concatena) y `*` **repite**:

```
print("Hola, " + "Ana")
print("3" + "4")      # texto + texto: los une, no los suma
print("Ja" * 3)
```

SALIDA

```
Hola, Ana
34
JaJaJa
```

7 Leer errores sin miedo

Equivocarse es parte de programar, también para quienes llevan años haciéndolo. Un error (en inglés, *bug*) **no es un fracaso: es el computador diciéndote dónde mirar**. Buscar y corregir errores se llama **depurar**.

Anatomía de un mensaje de error

```
NameError                 Traceback (most recent call last)
----> 1 print(Hola)

NameError: name 'Hola' is not defined
```

1. **Qué tipo de error es:** la primera palabra (`NameError`).
2. **Dónde:** la flecha `---->` marca la línea que falló.
3. **Por qué:** la explicación, en la **última línea**.

> **CONSEJO**
> Empieza siempre por la **última línea** del mensaje: ahí está la pista principal. Antes de corregir, intenta adivinar la causa.

Los tres errores más comunes

ERROR	CÓDIGO CON ERROR	QUÉ SIGNIFICA	CORRECCIÓN
`NameError`	`print(Hola)`	Python no conoce ese nombre. Aquí faltan las comillas, así que cree que `Hola` es una variable.	`print("Hola")`
`SyntaxError`	`print("Hola"`	Algo está mal escrito. Aquí falta cerrar el paréntesis: `'(' was never closed`.	`print("Hola")`
`TypeError`	`edad = "20"` `edad + 1`	La operación no sirve para ese tipo de dato: no se puede sumar texto con número.	`int(edad) + 1`

Otros errores que vas a encontrar

- `Print("hola")` → `NameError`: Python distingue mayúsculas, y la función se llama `print`.
- `print("Hola)` → `SyntaxError: unterminated string literal`: falta cerrar las comillas.
- `5 / 0` → `ZeroDivisionError: division by zero`: no se puede dividir entre cero.

Según la versión de Python, el texto de algunos mensajes puede cambiar un poco, pero el tipo de error es el mismo.

8 Pedir datos con `input()`

DEFINICIÓN

`input()` muestra un mensaje, espera a que el usuario escriba algo y presione Enter, y **devuelve lo que escribió** para que lo guardes en una variable.

```
nombre = input("¿Cómo te llamas? ")
print("Hola,", nombre)
```

SALIDA

```
¿Cómo te llamas? Ana
Hola, Ana
```

OJO

`input()` **siempre devuelve texto** (`str`), aunque el usuario escriba un número. Para hacer cálculos, conviértelo primero con `int()` o `float()` .

```
edad = int(input("¿Cuántos años tienes? "))
print("El próximo año tendrás", edad + 1)
```

SALIDA

```
¿Cuántos años tienes? 20
El próximo año tendrás 21
```

Tres ideas clave

1. Un programa es una lista de instrucciones que se ejecuta en orden.
2. Las variables guardan datos, y cada dato tiene un tipo.
3. Los errores son pistas: leerlos es parte de programar.

Glosario

Algoritmo	Secuencia finita y ordenada de pasos para resolver un problema.
Asignación	Guardar un valor en una variable con `=`.
Bug	Error en un programa.
Celda	Bloque de un notebook. Puede ser de código o de texto.
Comentario	Texto después de `#` que Python ignora. Sirve para explicar el código.
Concatenar	Unir textos con `+`.
Depurar	Buscar y corregir errores.
Expresión	Combinación de valores, variables y operadores que produce un resultado.
Función	Instrucción con nombre que hace una tarea, como `print()` o `type()`.
Intérprete	Programa que lee y ejecuta el código línea por línea.
Lenguaje de programación	Idioma con vocabulario y sintaxis para dar instrucciones al computador.
Notebook	Documento con celdas de código y de texto (archivo `.ipynb`).
Operador	Símbolo que hace una operación, como `+` o `**`.
Programa	Lista de instrucciones que el computador ejecuta en orden.
Salida	Lo que el programa muestra al ejecutarse.
Sintaxis	Reglas de escritura de un lenguaje.
Tipo de dato	Clase de valor (`int`, `float`, `str`, `bool`) que define qué se puede hacer con él.
Traceback	Mensaje de error de Python que indica dónde y por qué falló el código.
Variable	Nombre que guarda un dato para usarlo después.

Ejercicios

Resuélvelos en Colab antes de mirar las soluciones. Avanza hasta donde llegues.

Ponte a prueba: ¿qué imprime cada línea?

Escribe tu respuesta **antes** de ejecutar el código.

CÓDIGO	TU RESPUESTA
`print(7 // 2, 7 % 2)`	
`print("3" + "4")`	
`print(10 / 2)`	
`print(2 ** 3)`	
`print(type("True"))`	

Encuentra el error

Di qué tipo de error produce cada línea y corrígela.

CÓDIGO	ERROR Y CORRECCIÓN
`nota final = 4.5`	
`print("Hola)`	
`print(nombre)` (sin haber creado `nombre`)	

Ejercicios de la clase

BÁSICO **1.** Crea variables con tu nombre, tu carrera y tu semestre, e imprímelas en una frase.

BÁSICO **2.** Calcula cuántos minutos tiene una semana.

INTERMEDIO **3.** Una compra tiene 3 productos con precios distintos. Calcula el total y el total con IVA del 19 %.

INTERMEDIO **4.** Usa `type()` para verificar el tipo de cada variable que creaste.

RETO **5.** Convierte una temperatura de grados Celsius a Fahrenheit (`F = C * 9/5 + 32`) y muestra el resultado con un mensaje claro.

RETO **6.** Usa `input()` para pedir el nombre del usuario y saludarlo. ¿Qué tipo de dato devuelve `input()`?

Soluciones

Hay muchas formas correctas de resolver cada ejercicio. Si tu código es distinto pero da el resultado correcto, también está bien.

Ponte a prueba

CÓDIGO	SALIDA	POR QUÉ
`print(7 // 2, 7 % 2)`	`3 1`	7 entre 2 da 3 y sobra 1.
`print("3" + "4")`	`34`	Son textos: `+` los une.
`print(10 / 2)`	`5.0`	`/` siempre da un `float`.
`print(2 ** 3)`	`8`	2 × 2 × 2.
`print(type("True"))`	`<class 'str'>`	Está entre comillas: es texto, no `bool`.

Encuentra el error

CÓDIGO	ERROR	CORRECCIÓN
`nota final = 4.5`	`SyntaxError`: el nombre tiene un espacio.	`nota_final = 4.5`
`print("Hola)`	`SyntaxError`: faltan las comillas de cierre.	`print("Hola")`
`print(nombre)`	`NameError`: la variable no existe todavía.	Crear antes `nombre = "Ana"`

Próxima sesión: condicionales. ¿Y si el programa pudiera tomar decisiones? Trae tus dudas de esta guía a la clase o escríbelas en el Moodle FCE.